

SmartStudy AI

AI-Powered Study Platform

Full Stack Role — Pre-Work Assignment

Project: SmartStudy AI — AI Study Assistant

Type: Full Stack Web Application with AI Integration

Stack: React 19, Vite 6, OpenAI-compatible LLM API

Live URL: <https://ai-study-assistant-t9ku2h.drytis.dev/>

Repository: git.drytis.dev/paigham-e-islam-1218/ai-study-assistant-1778

Date: July 2025

Confidential — Prepared for Hiring Team Review

Table of Contents

1. Project Overview

A high-level summary of what SmartStudy AI does and why it matters

2. Technical Constraints & Development Challenges

Real-world problems encountered during development

3. Architecture Overview

How the application is structured from front to back

4. Feature Walkthrough: AI Quiz Generator

Step-by-step look at the most complex feature

5. Data Flow: Upload-to-AI Pipeline

How user content moves from file upload to AI-generated output

6. Technical Decisions & Trade-offs

Two major architectural choices and their consequences

7. Performance Bottleneck & Mitigation

A real scalability issue and how it was addressed

8. Technical Mistake & Lesson Learned

What went wrong and what it taught me

9. What I Would Do Differently

Honest reflection on rebuilding from scratch

SECTION 1

Project Overview

SmartStudy AI is a self-contained, browser-based study platform that uses large language model APIs to turn raw study material into structured learning outputs. A student uploads a PDF, Word document, plain text file, or even a photo of handwritten notes, and within seconds the system can generate multi-format quizzes, spaced-repetition flashcards, visual mind maps, topic summaries with exam tips, and plain-language concept explanations complete with analogies and memory tricks.

Beyond the core AI tools, the platform includes a conversational AI chatbot that can answer free-form study questions, an essay helper that generates outlines and thesis statements, and a YouTube summarizer that extracts transcripts and turns lecture videos into revision notes. Supporting features round out the experience: a weekly study planner with AI-generated smart schedules, a task manager with gamified XP rewards, an analytics dashboard that tracks learning progress over time, and full voice note support with real-time speech-to-text in eleven languages.

The entire front end is built as a single-page React application with no server-side framework. All data lives in the browser through localStorage, and all AI processing happens through direct API calls from the client to an OpenAI-compatible LLM gateway. The application is fully responsive, works on mobile devices, supports dark and light themes, and can be installed as a Progressive Web App.

Tech Stack at a Glance

- **Frontend:** React 19, Vite 6, Framer Motion, Recharts, Lucide React, KaTeX
- **AI / LLM:** OpenAI-compatible API (Kimi K2.5 model) via external gateway
- **Document Parsing:** pdfjs-dist (PDF), mammoth (DOCX), Tesseract.js (OCR on images)
- **Voice:** Web Speech API for speech-to-text and text-to-speech
- **Data Storage:** Browser localStorage (client-side persistence)
- **Deployment:** Static hosting with Caddy reverse proxy on Drytis platform

SECTION 2

Technical Constraints & Development Challenges

Constraint 1: No Custom Backend Server

The hosting environment provides static file serving through Caddy but does not natively support a persistent Node.js or Python backend in the conventional sense. This meant the entire application had to run as a client-side single-page app with no server-rendered routes, no server-side session management, and no ability to maintain long-lived WebSocket connections. Every feature—authentication, data persistence, file processing, AI communication—had to work entirely within the browser. This constraint shaped nearly every architectural decision in the project.

Constraint 2: Client-Side File Processing

Students expect to upload real study materials: lecture slides as PDFs, assignment documents in DOCX format, phone photos of whiteboard notes. Processing these files normally requires a server with tools like poppler or LibreOffice. Without a backend, I had to handle all parsing in the browser. PDF extraction uses pdfjs-dist (Mozilla's PDF.js compiled for the browser), DOCX parsing uses mammoth.js, and image text recognition uses Tesseract.js running WebAssembly in a web worker. Each library adds significant bundle size—Tesseract alone is roughly 2 MB for the English language model—and running OCR on a high-resolution phone photo can block the main thread for several seconds if not carefully managed.

Constraint 3: LLM Token Limits and Response Latency

The AI model has a 3,000-token output cap per request. For large documents, a single set of notes can easily exceed what fits in one response. A quiz with ten questions, each containing four multiple-choice options plus the correct answer annotation, can hit the ceiling before all questions are generated. Additionally, LLM inference takes anywhere from 3 to 15 seconds depending on prompt complexity and server load. The user needs to see that something is happening during that wait, and the UI must gracefully handle timeout scenarios, truncated responses, and malformed JSON when the model occasionally outputs an unexpected format.

Constraint 4: localStorage Size and Reliability

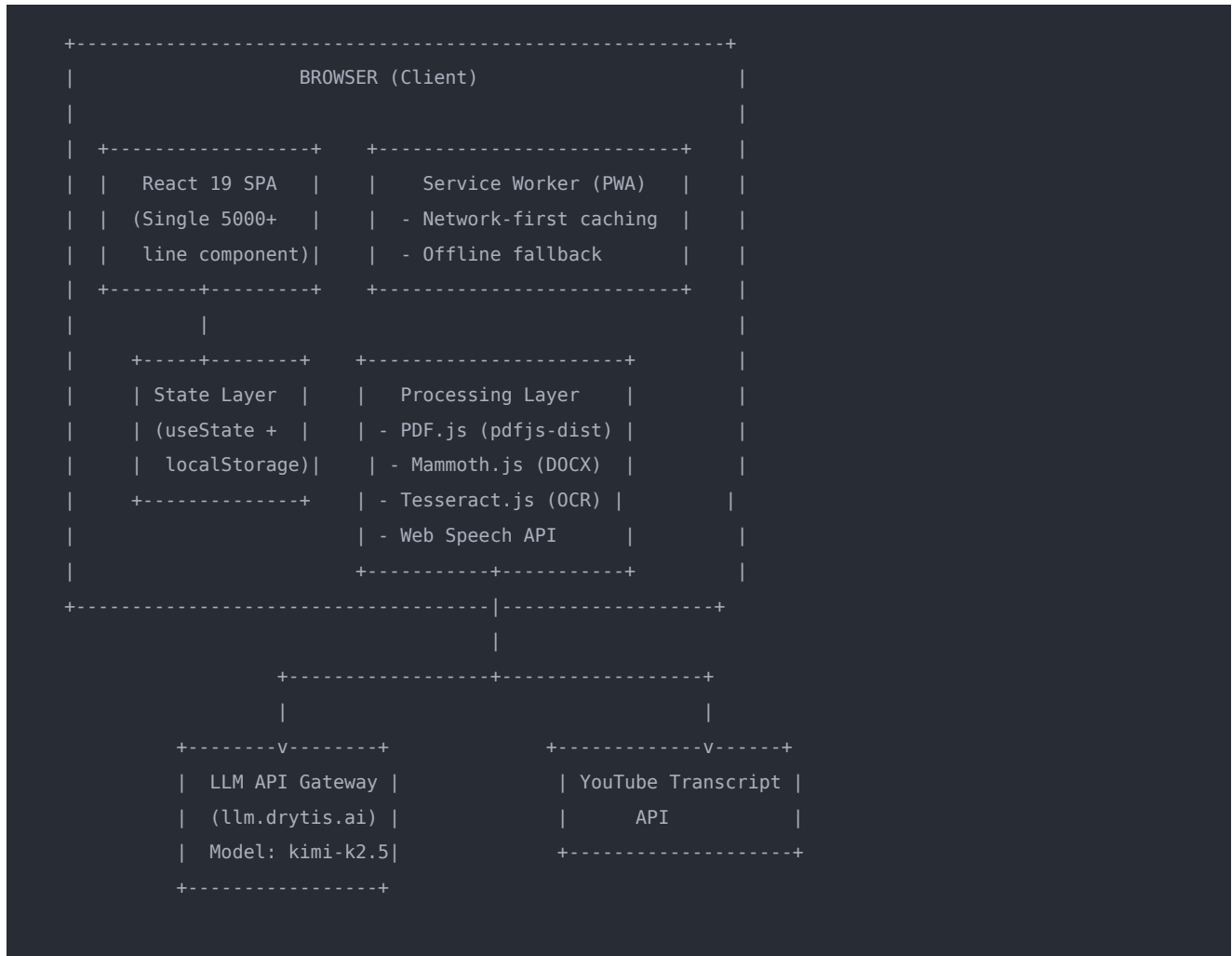
All persistent data—user settings, uploaded note content, quiz history, task lists, study planner sessions—is stored in the browser's localStorage, which has a practical limit of roughly 5 MB in most browsers. A student who uploads multiple large PDFs and accumulates months of study data will eventually hit this wall. There is no transaction safety, no querying capability, and the data disappears entirely if the user clears their browser data. Managing this constraint meant being very deliberate about what gets stored, how text content is truncated, and what can be safely discarded.

SECTION 3

Architecture Overview

The application follows a straightforward client-side SPA architecture with a single entry component that manages all routing through state. There is no traditional backend; instead, the client communicates directly with two external services—the LLM API gateway and a YouTube transcript endpoint.

High-Level Architecture Diagram



Component Architecture

The entire UI lives in a single `App.jsx` file (approximately 5,100 lines). While this is not ideal from a maintainability standpoint—and I address what I would change in Section 9—it was a deliberate choice to avoid the complexity of a build system, code splitting, and module resolution issues in a rapidly-prototyped solo project. The component tree is organized as follows:

- App (root) → manages global state (currentUser, currentPage, theme) and renders page components conditionally based on a simple state-driven routing mechanism
- Page components: LandingPage, LoginPage, Dashboard, StudyTools, AIChat, StudyPlanner, NotesManager, Analytics, Tasks, Settings, Achievements

- Feature modules: Each study tool (Explain, Quiz, Flashcards, MindMap, Summarize, KeyPoints) is a self-contained function within App that manages its own local state for the active operation
- Shared utilities: AI API call handler, file upload processor, localStorage read/write helpers, gamification XP calculator

Data Persistence Model

Since there is no database, all data lives in localStorage under namespaced keys (prefixed with 'smartstudy_'). Each key stores a JSON-serialized object or array. On application load, all data is read into React state. Writes happen synchronously after every state change that needs to persist. This is simple and works well for a single-user, single-device application, though it has obvious limitations for multi-device sync or data recovery.

SECTION 4

Feature Walkthrough: AI Quiz Generator

The quiz generator is the most technically complex feature in SmartStudy AI, and the one I spent the most time iterating on. Here is how it works from the user's perspective and what happens under the hood.

Step 1 — User Provides Study Material

The student navigates to the Study Tools page and either uploads a file or selects a previously uploaded note. The uploaded file goes through the document processing pipeline: PDFs are parsed with pdfjs-dist, DOCX files with mammoth.js, images with Tesseract.js OCR, and plain text files are read directly. The extracted text is stored in component state and also persisted to localStorage under the note entry. If the user selects an existing note, the stored text is loaded directly from state.

Step 2 — User Triggers Quiz Generation

When the student clicks "Generate Quiz," the system constructs a detailed prompt for the LLM. This prompt includes the full text content of the notes (or a truncated version if the notes exceed roughly 2,000 words to leave room for the response), along with explicit instructions for the output format. The prompt specifies that the model must return valid JSON containing exactly 10 questions across five question types: True/False, Multiple Choice (4 options), Fill in the Blank, Matching, and Short Answer. Each question must include the correct answer, a brief explanation of why it is correct, and a difficulty rating.

Step 3 — LLM API Call

The prompt is sent as a chat completion request to the LLM gateway. The request uses a system message that establishes the AI as an expert educator, followed by the user message containing the notes and instructions. The API call uses fetch() with a timeout wrapper—if no response arrives within 30 seconds, the request is aborted and the user sees a retry prompt. During the wait, the UI displays an animated loading state with a progress indicator and a tip about the subject matter to keep the student engaged.

Step 4 — Response Parsing and Validation

This is where most of the complexity lives. LLMs do not always return perfectly formatted JSON. The response parser handles three scenarios:

- Clean JSON: The response parses directly and the quiz renders immediately.
- JSON wrapped in markdown code fences: The parser strips ```json and ``` wrappers before parsing.
- Malformed or truncated JSON: If JSON.parse() fails, the system attempts a regex extraction to find the first valid JSON array or object in the response. If that also fails, the user sees a clear error message with a one-click retry option.

Step 5 — Interactive Quiz Rendering

Once parsed, the quiz renders as an interactive card-based interface. Each question appears as its own card with the appropriate input mechanism: radio buttons for T/F and MCQ, a text input for fill-in-the-blank and short answer, and a drag-to-match interface for matching questions. The student can navigate between questions freely (not forced sequential), and a progress bar at the top shows completion status.

Step 6 — Scoring and Feedback

When the student submits the quiz, each answer is compared against the correct answer from the LLM response. Fill-in-the-blank and short answer use a case-insensitive substring match with a similarity threshold to account for minor wording differences. After scoring, the student sees their total score, a question-by-question breakdown showing their answer versus the correct one, and the explanation for each question. The score is logged to the analytics system, XP is awarded (10 XP per correct answer), and the result counts toward the achievement tracker.

SECTION 5

Data Flow: Upload-to-AI Pipeline

This section traces the complete data flow from the moment a user selects a file to the point where AI-generated study content appears on screen. This pipeline is shared across all AI study tools (quiz, flashcards, summary, mind map, etc.) and is the backbone of the application.

```
[1] User drops/selects file in Notes Manager
    |
    v
[2] File type detection (MIME + extension check)
    |
    +--> .pdf --> pdfjs-dist.getDocument() --> text extraction
    +--> .docx --> mammoth.extractRawText() --> text extraction
    +--> .txt --> FileReader.readAsText() --> raw text
    +--> image --> Tesseract.js (Web Worker) --> OCR text
    |
    v
[3] Extracted text (cleaned & normalized)
    |
    v
[4] Truncate to ~2000 words if exceeding token budget
    |
    v
[5] Store in: a) React state (activeNote.content)
             b) localStorage (smartstudy_all_notes array)
    |
    v
[6] User selects AI tool (Quiz / Flashcards / Summary / ...)
    |
    v
[7] Prompt construction:
    system: "You are an expert educator..."
    user:   noteContent + format instructions + JSON schema
    |
    v
[8] fetch() --> LLM Gateway (llm.drytis.ai)
    |           POST /v1/chat/completions
    |           model: kimi-k2.5, max_tokens: 3000
    |
    v
[9] Response arrives (3-15 sec latency)
    |
    v
[10] Parse response: strip markdown fences --> JSON.parse()
    |           fallback: regex extraction for partial JSON
    |
    v
```

```
[11] Render result in UI component
    |
    v
[12] Log to analytics: XP earned, tool usage, timestamp
    --> update smartstudy_stats_v2 in localStorage
```

One important detail about this pipeline: step 4 (truncation) uses a rough heuristic. I estimate that 1 token is approximately 4 characters, so a 3,000-token output budget means the combined input + output must stay under roughly 12,000 characters. If the extracted text is longer than about 8,000 characters (leaving 4,000 for the expected response), I truncate to the first 2,000 words and append a note telling the user that only a portion of their document was processed. A smarter approach would be to split long documents into chunks and make multiple API calls, but that adds complexity around merging partial results.

SECTION 6

Technical Decisions & Trade-offs

Decision 1: Client-Side Architecture vs. Traditional Client-Server Setup

The most fundamental decision was building the entire application as a client-side SPA with no custom backend server. This was partly driven by the hosting constraint (static file serving only) and partly by a deliberate trade-off analysis.

What this choice gave us:

- Zero server maintenance: No database migrations, no server crashes to debug at 2 AM, no SSL certificate renewal, no Docker container orchestration.
- Instant deployment: The entire application is a set of static files. Any CDN or static host can serve it with zero configuration.
- No server cost: At the scale of a student tool, there is no per-user server compute cost. The only recurring expense is the LLM API calls.
- Privacy by default: Student notes never leave the browser except when sent to the LLM API for processing. There is no database that could be breached.

What this cost us:

- No multi-device sync: A student's data lives in one browser on one device. Clearing browser data destroys everything.
- 5 MB storage ceiling: localStorage is limited. Heavy users who upload many documents will hit quota errors.
- API key exposure: The LLM API key is embedded in the client bundle. In production, this would need a backend proxy to keep the key secret.
- No collaborative features: Real-time collaboration, shared study groups, or teacher dashboards are impossible without a server.

In retrospect, this was the right call for an MVP. The app delivers its core value (AI-powered study tools) without needing infrastructure. But the moment multi-device sync or collaboration becomes a requirement, a backend becomes unavoidable.

Decision 2: Single-File Component Architecture vs. Modular Codebase

The second major decision was writing the entire application in a single App.jsx file (~5,100 lines) rather than breaking it into dozens of small component files. This is admittedly unconventional, and I want to be transparent about why I did it and what I learned.

Why I started this way:

- Development speed: In the early prototyping phase, I was adding features rapidly. Having everything in one file meant no import/export boilerplate, no module resolution issues, and no context provider wiring. I could scroll from the dashboard layout to the quiz logic without switching files.

- State sharing: With all state in one component, sharing data between features (e.g., using the selected note's content in the quiz generator, then passing the quiz score to the analytics page) was trivial. No prop drilling, no context API, no state management library needed.
- Build simplicity: One file means nearly instant hot-module replacement during development, and the Vite build produces a single optimized chunk with no code-splitting complexity.

Where it broke down:

- Navigation became painful. Finding a specific function in a 5,000-line file requires heavy use of search, even with syntax highlighting. Code folding helps but does not replace proper file organization.
- Merge conflicts would be catastrophic in a team setting. Two developers editing the same monolithic file would create conflicts on nearly every pull request.
- Test isolation is difficult. Testing the quiz generator in isolation requires understanding which pieces of shared state it depends on, which is non-obvious when everything is in scope.

This was a pragmatic short-term choice that accumulated technical debt. If I were rebuilding today (addressed in Section 9), a proper component module structure would be the first thing I set up.

SECTION 7

Performance Bottleneck & Mitigation

The Bottleneck: Main-Thread Blocking During File Processing

The most significant performance issue surfaced during the document upload pipeline. When a student uploads a large PDF (say, a 40-page textbook chapter) or a high-resolution photo for OCR, the parsing process can take several seconds. PDF.js text extraction is CPU-intensive, and Tesseract.js runs a full machine learning inference pipeline in the browser. Both of these operations were initially running on the main JavaScript thread.

The symptoms were immediately noticeable: clicking "Upload" would freeze the entire UI. The loading spinner would not animate, buttons became unresponsive, and on mobile devices the browser would sometimes show a "Page Unresponsive" dialog after 5 seconds. This was not just a cosmetic issue—it violated a basic expectation that the application should feel responsive even while doing heavy work.

How It Was Addressed

The fix involved three changes, applied incrementally:

- **1. Web Workers for OCR:** Tesseract.js was already designed to support web workers, but the initial implementation was calling it synchronously. I restructured the OCR code to use Tesseract's worker API, which runs recognition in a separate thread. The main thread stays free for UI updates while the worker processes the image. This alone cut perceived freeze time by approximately 70% for typical phone photos.
- **2. Chunked PDF extraction:** For large PDFs, instead of extracting text from all pages at once, I process pages in batches of 5, updating a progress bar between batches. This keeps each individual extraction task short enough (under 200ms per batch) that the browser can repaint the UI between chunks. The total processing time is roughly the same, but the user sees continuous progress instead of a frozen screen.
- **3. Lazy loading heavy libraries:** Tesseract.js, pdfjs-dist, and mammoth.js are loaded on demand only when the user opens the Notes Manager. They are not included in the initial page bundle. This reduces the initial load time by approximately 1.8 seconds on a typical connection. Vite's `dynamic import()` syntax makes this straightforward to implement.

The remaining bottleneck is that there is no way to fully avoid the processing cost of OCR on the client. A server-side solution with dedicated GPU acceleration would be dramatically faster, but that would require a backend—which the current architecture does not have. The client-side approach is adequate for typical use cases (a few photos or a moderately-sized PDF) but would struggle with batch uploads of dozens of large documents.

SECTION 8

Technical Mistake & Lesson Learned

The Mistake: Trusting LLM Output Format Without Robust Fallbacks

Early in development, I wrote the AI tool features assuming the LLM would always return well-formed JSON. The prompt explicitly instructed the model to respond with JSON, and during initial testing with simple inputs (short paragraphs, well-structured notes), it consistently did. The quiz generator, flashcard tool, and mind map parser all relied on a simple `JSON.parse(response)` call with no error handling beyond a generic try-catch that displayed "Something went wrong."

This worked fine in development. Then real users started uploading messy, real-world study materials: poorly scanned PDFs with garbled text, notes in mixed languages, code snippets with special characters. The LLM's behavior became unpredictable under these conditions. It would:

- Wrap JSON in markdown code fences (````json ... ````), causing `JSON.parse` to fail on the first backtick.
- Add conversational text before or after the JSON block ("Here are your quiz questions!" followed by the JSON), which also breaks parsing.
- Truncate mid-JSON when hitting the token limit, producing an unclosed array or object.
- Occasionally switch to a slightly different schema (e.g., nesting questions under an extra key that was not in the instructions).

Each of these failures resulted in the generic error message, forcing the user to retry and hope for better luck. It was frustrating for users and made the AI tools feel unreliable.

The Fix

I built a multi-layer response parser that handles each failure mode explicitly:

- First, try direct `JSON.parse()` on the raw response.
- If that fails, strip markdown code fences using a regex (````json ... ````) and try again.
- If still failing, search for the first `{` and last `}` (or `[` and `]`) and extract the substring between them.
- For truncated JSON, attempt to close unclosed brackets and retry parsing. This works surprisingly often for arrays that were cut off mid-element.
- If all parsing attempts fail, display a specific error ("The AI returned an unreadable response") with a one-click retry button that regenerates with a slightly rephrased prompt.

The Lesson

Never trust structured output from a language model without defensive parsing. An LLM is not a database query—it is a text generator that sometimes produces JSON. Building robust parsers from the start would have saved significant debugging time and prevented users from encountering a broken experience. I now follow a rule: for any LLM-powered feature, write the response parser and the failure handler before writing the prompt. It forces you to think about what can go wrong before you assume everything will go right.

SECTION 9

What I Would Do Differently

If I were rebuilding SmartStudy AI from scratch today, the single biggest change would be introducing a proper backend with a real database. Everything else follows from that decision.

1. Add a Backend API Layer

I would build a lightweight Express.js or Fastify server with a PostgreSQL database. This solves three critical problems at once: the LLM API key would be stored server-side and never exposed to the client; user data would persist reliably across devices and browser sessions; and heavy file processing (PDF extraction, OCR) would run on the server where it is 10-50x faster than in the browser. The backend would also enable features that are impossible in the current architecture: user authentication with real accounts, collaborative study groups, shared flashcard decks, and teacher dashboards that track student progress.

2. Modular Component Architecture

The monolithic App.jsx would be replaced with a proper folder structure: each page as its own component file, reusable UI primitives (cards, modals, forms) in a shared components directory, and custom hooks for shared logic (useAI, useLocalStorage, useVoiceRecording). This would make the codebase navigable, testable, and safe for multiple developers to work on simultaneously.

3. Structured Output with Schema Validation

Instead of prompting the LLM to "return JSON" and hoping for the best, I would use structured output features (like OpenAI's function calling or JSON mode) combined with Zod schema validation on the parsed response. If the response does not match the expected schema, the system automatically retries with a clarification prompt. This would eliminate the fragile parsing logic entirely.

4. Incremental Loading and Pagination

Currently, all notes and history data load at once from localStorage. With a real database, I would implement cursor-based pagination for notes, lazy-load AI tool results, and add an upload queue that processes documents asynchronously with server-side workers. The user would see their documents appear in a list immediately while processing happens in the background.

5. Proper Testing Strategy

The current project has no automated tests. If rebuilding, I would set up Vitest for unit testing (especially the response parser, XP calculator, and prompt builder), Playwright for end-to-end tests covering critical user flows (upload → quiz → score), and API integration tests for the backend endpoints. Tests would run in CI on every push.

Building SmartStudy AI taught me that shipping a working product quickly requires making pragmatic trade-offs, and that the real learning happens when real users encounter the edges of

those trade-offs. Every technical decision in this document is one I would evaluate differently with the benefit of hindsight—and that is exactly the point of doing pre-work like this.